Experiment Number : 06 **Date:**

ABSTRACT CLASSES

Aim:

To understand and implement inheritance concepts in Java.

PRE LAB EXERCISE

QUESTIONS

✓ What is an abstract class?

An abstract class is a class that cannot be created as an object.

It is used as a base class and may contain:

- Abstract methods (without body)
- Normal methods (with body)
- Variables and constructors

Purpose: To provide a common structure for other classes.

✓ Why are abstract methods used?

Abstract methods are used to declare a method without implementation.

The child class must provide the implementation.

Purpose:

- To ensure all child classes follow the same method structure
- To achieve abstraction (hide implementation details)

✓ Difference between abstract class and interface.

Abstract Class	Interface
Can have abstract and normal methods	Contains only abstract methods (default/public methods allowed)
Supports constructors and variables	No constructors, only constants
A class can extend only one abstract class	A class can implement multiple interfaces
Uses extends keyword	Uses implements keyword

IN LAB EXERCISE

Objective:

To implement abstract class and demonstrate abstraction.

PROGRAMS:

1.University System

Scenario:
A university has different types of courses: Online, Offline, and Hybrid. Each course has a getDetails() method.

Question:
Create an abstract class Course with abstract method getDetails(). Implement OnlineCourse, OfflineCourse, and HybridCourse classes.

Code:

```
abstract class Course {
    abstract void getDetails();
}


class OnlineCourse extends Course {
    void getDetails() {
        System.out.println("Online Course: Attend via Internet");
    }
}


class OfflineCourse extends Course {
    void getDetails() {
        System.out.println("Offline Course: Attend in classroom");
    }
}


class HybridCourse extends Course {
    void getDetails() {
```

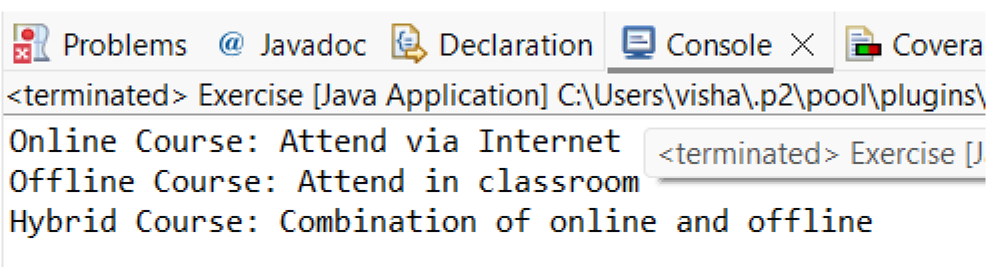
```
        System.out.println("Hybrid Course: Combination of online and offline");
    }
}

public class Main {
    public static void main(String[] args) {
        Course c1 = new OnlineCourse();
        Course c2 = new OfflineCourse();
        Course c3 = new HybridCourse();

        c1.getDetails();
        c2.getDetails();
        c3.getDetails();
    }
}
```

Output:

Online Course: Attend via Internet

Offline Course: Attend in classroom

Hybrid Course: Combination of online and offline



2. Employee Payroll System

Scenario:
A company has different types of employees — Regular and Contract. All employees have a salary, but the calculation differs for each type.

Question:
Design an abstract class Employee with an abstract method calculateSalary(). Implement subclasses RegularEmployee and ContractEmployee to calculate salary differently.

Code:

```
import java.util.Scanner;

abstract class Employee {
    String name;
    double baseSalary;

    // Abstract method to calculate total salary
    abstract void calculateSalary();
}

class RegularEmployee extends Employee {
    double bonusRate = 0.1; // 10% bonus

    void calculateSalary() {
        double totalSalary = baseSalary + (baseSalary * bonusRate);
        System.out.println("Regular Employee: " + name);
        System.out.println("Base Salary: " + baseSalary);
        System.out.println("Total Salary (with 10% bonus): " + totalSalary);
    }
}

class ContractEmployee extends Employee {
    void calculateSalary() {
        System.out.println("Contract Employee: " + name);
        System.out.println("Total Salary: " + baseSalary);
    }
```

```java
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input for Regular Employee
        System.out.print("Enter Regular Employee Name: ");
        String regName = sc.nextLine();
        System.out.print("Enter Base Salary: ");
        double regSalary = sc.nextDouble();
        sc.nextLine(); // Consume newline

        // Input for Contract Employee
        System.out.print("Enter Contract Employee Name: ");
        String conName = sc.nextLine();
        System.out.print("Enter Base Salary: ");
        double conSalary = sc.nextDouble();

        // Create objects
        Employee e1 = new RegularEmployee();
        e1.name = regName;
        e1.baseSalary = regSalary;

        Employee e2 = new ContractEmployee();
        e2.name = conName;
        e2.baseSalary = conSalary;
```

```
        System.out.println("\n--- Salary Details ---");
        e1.calculateSalary();
        System.out.println();
        e2.calculateSalary();

        sc.close();
    }
}
```

Output:

Enter Regular Employee Name: Ram

Enter Base Salary: 30000

Enter Contract Employee Name: Ravi

Enter Base Salary: 20000

--- Salary Details ---

Regular Employee: Anitha

Base Salary: 30000.0

Total Salary (with 10% bonus): 33000.0

Contract Employee: Ravi

Total Salary: 20000.0

```
<terminated> Newex [Java Application] C:\Users\visha\.p2\pc
--- Salary Details ---
Regular Employee: vishalatchi
Base Salary: 10000.0
Total Salary (with 10% bonus): 11000.0

Contract Employee: marushika
Total Salary: 20000.0
```

```
<terminated> Newex [Java Application] C:\Users\visha\.p2\
Enter Regular Employee Name: vishalatchi
Enter Base Salary: 10,000
Enter Contract Employee Name: marushika
Enter Base Salary: 20,000
```

3.Banking System

Scenario:

A bank has different types of accounts: Savings and Current. Both accounts need a method to calculate interest, but the calculation differs for each account type.

Question:

Use an abstract class BankAccount with an abstract method calculateInterest() and implement it in SavingsAccount and CurrentAccount classes.

Code

```
abstract class BankAccount {
    String accountHolder;
    double balance;

    BankAccount(String name, double bal) {
        accountHolder = name;
        balance = bal;
    }

    abstract void calculateInterest();  // Abstract method
}

class SavingsAccount extends BankAccount {
    double interestRate = 0.04; // 4% interest

    SavingsAccount(String name, double bal) {
        super(name, bal);
```

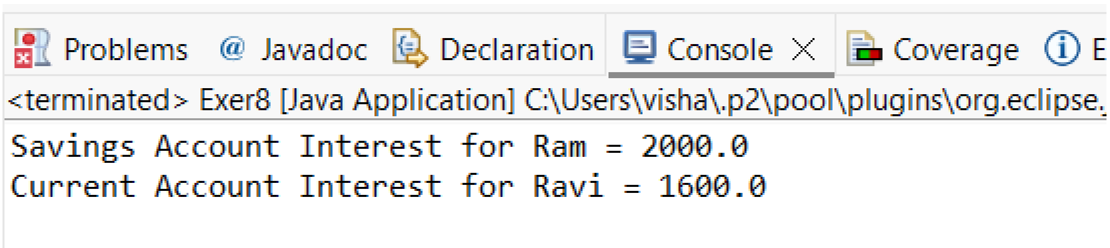
```
    }

    void calculateInterest() {
        double interest = balance * interestRate;
        System.out.println("Savings Account Interest for " + accountHolder + " = " + interest);
    }
}

class CurrentAccount extends BankAccount {
    double interestRate = 0.02; // 2% interest

    CurrentAccount(String name, double bal) {
        super(name, bal);
    }

    void calculateInterest() {
        double interest = balance * interestRate;
        System.out.println("Current Account Interest for " + accountHolder + " = " + interest);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc1 = new SavingsAccount("Ram", 50000);
        BankAccount acc2 = new CurrentAccount("Ravi", 80000);

        acc1.calculateInterest();
        acc2.calculateInterest();
```

```
    }
}
```

Output

Savings Account Interest for Ram = 2000.0

Current Account Interest for Ravi = 1600.0



POST LAB EXERCISE

- ✓ How is an abstract class different from a regular class?

Abstract Class	Regular Class
Cannot be instantiated (no objects)	Objects can be created
May contain abstract methods (no body)	Contains only implemented methods
Used as a base class for inheritance	Used to create objects directly

- ✓ Can you create an object of an abstract class? Why or why not?

  No.
  Reason:
  An abstract class may contain abstract methods without implementation.
  Since it is incomplete, Java does not allow creating its object.

- ✓ What happens if a subclass does not implement an abstract method?

  The subclass must be declared as abstract.

  Otherwise, Java will give a compile-time error.

- ✓ Can an abstract class exist without any abstract methods?

Yes.
An abstract class can have only normal methods.
It is declared abstract to prevent object creation and to be used only for inheritance.

- ✓ Can an abstract class extend another abstract class?

Yes.
An abstract class can extend another abstract class.
The child class can implement the abstract methods or remain abstract.

Result:

Thus the abstract classes and methods were implemented and executed successfully.

ASSESSMENT

Description	**Max Marks**	**Marks Awarded**
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
Faculty Signature		